



Desarrollo de Aplicaciones con APIs de IA (70 horas)



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

➤ Procesamiento de lenguaje natural
aplicado a software.



- I. ¿Qué permite el NLP en una aplicación?
- II. Casos de uso principales
- III. Flujo típico de NLP en una app
- IV. Ejemplo: análisis de sentimiento (Node.js)
- V. Ejemplo: extracción de entidades (Python)
- VI. Búsqueda semántica con embeddings
- VII. Tabla: ¿Qué técnica usar?
- VIII. Seguridad: buenas prácticas y errores comunes



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

Idea clave

El NLP permite que una aplicación entienda, procese y genere lenguaje humano.

Gracias a las APIs de IA modernas, cualquier software puede incorporar:

- Comprensión de texto
- Resumen
- Clasificación
- Traducción
- Extracción de información
- Conversación natural

Sin entrenar modelos propios.

Procesamiento de lenguaje natural aplicado a software.



Idea clave

Comprensión de texto Interpretan preguntas, instrucciones, intenciones y significado.

Ejemplo: detectar qué quiere un usuario en un mensaje de soporte.

Resumen Reducen textos largos a ideas clave manteniendo el contexto.

Ejemplo: convertir un informe de 10 páginas en 5 puntos esenciales.

Clasificación Asignan categorías, etiquetas o niveles a un texto.

Ejemplo: clasificar comentarios como positivos, negativos o neutros.



Idea clave

Traducción contextual Traducen manteniendo tono, intención y precisión técnica.

Ejemplo: traducir documentación sin perder matices.

Extracción de información Identifican datos concretos dentro de un texto.

Ejemplo: extraer nombres, fechas o entidades de un documento.

Conversación natural Generan respuestas coherentes y fluidas en formato chat.

Ejemplo: asistentes, chatbots, tutores virtuales.

Procesamiento de lenguaje natural aplicado a software.



■ Si en nuestra aplicación necesitamos realizar tareas complejas como:

- Leer y entender documentos
- Responder preguntas de usuarios
- Detectar emociones o intenciones
- Resumir información extensa
- Clasificar mensajes automáticamente
- Mantener conversaciones naturales

■ Tendrías que:

- Buscar y limpiar datasets
- Entrenar modelos enormes
- Ajustarlos continuamente
- Mantener servidores con GPUs
- Gestionar fallos, sesgos y actualizaciones

■ Si lo hicieras entrenando modelos propios...

Sería como crear un equipo completo desde cero:

- Lingüistas para entender el lenguaje
- Traductores para manejar varios idiomas
- Psicólogos para detectar emociones
- Analistas para extraer datos relevantes
- Redactores para generar respuestas coherentes
- Ingenieros para entrenar, ajustar y mantener los modelos

Es decir: mucho tiempo, mucho dinero y mucha infraestructura.



Procesamiento de lenguaje natural aplicado a software.



■ Con APIs de NLP modernas...

Es como contratar a ese equipo ya formado, disponible:

- 24/7
- Escalable a cualquier volumen
- Sin necesidad de formación
- Sin coste de infraestructura
- Sin mantenimiento
- Con mejoras automáticas del proveedor

■ Tu aplicación solo necesita hacer una petición y recibe:

- Texto comprendido
- Resúmenes claros
- Clasificaciones precisas
- Traducciones contextuales
- Datos extraídos
- Conversaciones naturales

Todo sin entrenar modelos, sin gestionar hardware y sin complicaciones.

El NLP mediante APIs convierte tareas complejas de IA en simples llamadas a funciones. No necesitas ser experto en machine learning para construir aplicaciones inteligentes.



2,1— ¿Qué permite el NLP en una aplicación?

- Entender texto
- Generar respuestas coherentes
- Resumir documentos
- Traducir contenido
- Detectar intención del usuario
- Extraer entidades (nombres, fechas, lugares)
- Clasificar mensajes o documentos
- Buscar por significado (búsqueda semántica)

2,1— ¿Qué permite el NLP en una aplicación?

Entender texto

La aplicación interpreta el significado del mensaje: intención, contexto, tono y contenido.

Ejemplo: distinguir si un usuario está preguntando, quejándose o solicitando ayuda.

Generar respuestas coherentes

Produce texto claro, natural y adaptado al contexto previo.

Ejemplo: responder a una duda técnica manteniendo el estilo de un asistente profesional.

Resumir documentos

Reduce textos extensos a sus ideas esenciales sin perder información crítica.

Ejemplo: transformar un informe de 15 páginas en un resumen de 5 puntos clave.

■ 2,1— ¿Qué permite el NLP en una aplicación?

Traducir contenido

Traduce manteniendo el significado, el tono y la precisión técnica.

Ejemplo: traducir documentación de soporte sin perder matices importantes.

Detectar intención del usuario

Identifica qué quiere hacer el usuario: pedir información, reportar un error, solicitar una acción, etc.

Ejemplo: clasificar un mensaje como "incidencia", "pregunta general" o "feedback".

Extraer entidades (nombres, fechas, lugares)

Encuentra datos concretos dentro de un texto sin necesidad de reglas manuales.

Ejemplo: extraer nombres de clientes, fechas de entrega o direcciones de un correo.

■ 2,1— ¿Qué permite el NLP en una aplicación?

Clasificar mensajes o documentos

Asigna categorías automáticas según tema, urgencia o tipo de contenido.

Ejemplo: etiquetar correos como "urgente", "facturación", "soporte técnico".

Buscar por significado (búsqueda semántica)

Permite encontrar información aunque no coincidan las palabras exactas.

Ejemplo: buscar "problemas al iniciar sesión" y encontrar documentos que hablan de "fallos de autenticación".

2,1— ¿Qué permite el NLP en una aplicación?

EL NLP ES MUCHO MÁS QUE "ENTENDER TEXTO"

Cuando hablamos de Procesamiento de Lenguaje Natural (NLP) en el contexto de desarrollo de software, no nos referimos solo a que la aplicación "lea" palabras. Nos referimos a capacidades que antes requerían equipos enteros de lingüistas, analistas e ingenieros de machine learning.

ENTENDER TEXTO NO ES SOLO LEER PALABRAS

Entender texto implica múltiples capas de análisis:

- **Análisis sintáctico:** ¿Qué estructura gramatical tiene la frase?
- **Análisis semántico:** ¿Qué significado tiene? ¿Hay ambigüedad?
 - **Análisis pragmático:** ¿Cuál es la intención del usuario?
¿Está preguntando, ordenando, quejando
- **Análisis de sentimiento:** ¿Qué emoción transmite?
¿Es positivo, negativo, neutral, sarcástico?

■ 2,1— ¿Qué permite el NLP en una aplicación?

Por ejemplo, la frase "Vaya, qué bien..." puede ser positiva o sarcástica según el contexto.

Un NLP avanzado puede distinguirlo.

GENERAR RESPUESTAS COHERENTES NO ES SOLO "DECIR ALGO"

Generar texto no es solo devolver una frase predefinida. Un buen sistema de NLP debe:

- Mantener coherencia con el historial de la conversación
- Adaptar el tono según el usuario (formal, coloquial, técnico)
- Evitar repetir información innecesariamente
- Saber decir "no sé" cuando no tiene información suficiente

2,2— Casos de uso principales



Caso de uso	Descripción ampliada	API recomendada
Chatbots	Conversación natural, memoria de contexto, respuestas coherentes y adaptadas al usuario. Ideal para asistentes, soporte y tutores.	OpenAI (GPT-4o / mini) por fluidez; Claude por análisis profundo; Gemini por multimodalidad.
Resumen	Reducción de documentos largos a ideas clave, resúmenes ejecutivos o estructurados. Funciona con PDFs, artículos, informes.	Claude (contexto largo), GPT-4o (resumen estructurado).
Análisis de sentimiento	Detecta emociones, tono, polaridad y actitud del texto. Útil en soporte, RRSS y análisis de clientes.	Hugging Face (modelos especializados), OpenAI (clasificación rápida).
Traducción	Traducción contextual manteniendo tono, intención y precisión técnica.	Google AI (Gemini) por multilingüe; GPT-4o para traducción técnica.
Extracción de entidades (NER)	Identifica nombres, fechas, lugares, organizaciones, números o conceptos clave dentro de un texto.	Hugging Face NER, spaCy, OpenAI para extracción flexible.
Clasificación	Asigna categorías a textos: temas, urgencia, tipo de documento, spam/no spam, etc.	Hugging Face BERT, OpenAI (clasificación zero-shot).
Búsqueda semántica	Busca por significado, no por palabras exactas. Ideal para RAG, documentación y FAQs.	OpenAI Embeddings, Cohere, Hugging Face.
Conversación multimodal	Combina texto + imagen para análisis, descripciones o interacción avanzada.	Gemini, GPT-4o, Claude 3.
Análisis de documentos	Lectura, extracción, resumen y estructuración de PDFs o textos largos.	Claude (contexto largo), GPT-4o.
Generación de contenido	Redacción de textos, guías, emails, explicaciones o contenido creativo.	OpenAI, Claude

■ 2,2— Casos de uso principales



1. SOPORTE TÉCNICO AUTOMATIZADO

Un chatbot de soporte que:

- Clasifica tickets automáticamente por categoría, prioridad y departamento responsable
- Resuelve consultas frecuentes sin intervención humana
- Escala a agentes humanos cuando detecta problemas complejos o urgencia alta
- Extrae información clave del usuario (número de pedido, producto, versión) para agilizar la resolución

Ejemplo real: Un usuario escribe "No puedo iniciar sesión después de actualizar la app". El sistema detecta intención (soporte técnico), categoría (cuenta/acceso), extrae contexto (actualización reciente) y puede ofrecer soluciones automáticas (restablecer contraseña, verificar versión, limpiar caché).



■ 2,2— Casos de uso principales

2. ANÁLISIS DE FEEDBACK Y RESEÑAS

Una plataforma de e-commerce que:

- Analiza miles de reseñas de productos automáticamente
- Extrae qué aspectos se mencionan positivamente (calidad, precio, envío rápido)
- Identifica quejas recurrentes (embalaje dañado, producto defectuoso)
- Calcula puntuaciones de sentimiento por categoría de producto
- Alerta sobre productos con tendencia negativa para tomar acciones

Ejemplo real: Una cadena de restaurantes analiza reseñas y descubre que la queja más frecuente no es la comida, sino el tiempo de espera. Puede optimizar procesos o ajustar expectativas.

■ 2,2— Casos de uso principales



3. BÚSQUEDA SEMÁNTICA EN DOCUMENTACIÓN

Un sistema de documentación técnica que:

- Permite buscar "cómo conectar la base de datos" y encuentra secciones sobre "configuración de conexión a PostgreSQL"
- Sugiere artículos relacionados aunque no compartan palabras clave
- Entiende sinónimos técnicos: "login" / "autenticación" / "inicio de sesión"
- Agrupa documentos por temas sin necesidad de etiquetas manuales

Ejemplo real: Un portal de ayuda de una empresa de software permite buscar "error 404" y muestra no solo artículos sobre ese error, sino también sobre "página no encontrada", "URL incorrecta", "enlace roto".

■ 2,2— Casos de uso principales



4. PROCESAMIENTO DE DOCUMENTOS LEGALES

Una aplicación para departamentos legales que:

- Resume contratos extensos extrayendo cláusulas clave
- Identifica fechas límite, obligaciones, penalizaciones
- Compara versiones de contratos detectando cambios significativos
- Extrae entidades como partes involucradas, montos, plazos

Ejemplo real: Un bufete de abogados procesa 50 contratos diarios, extrayendo automáticamente fechas de renovación y obligaciones de confidencialidad, reduciendo horas de trabajo manual.

■ 2,2— Casos de uso principales

5. ASISTENTES DE CÓDIGO

Herramientas de desarrollo que:

- Explican fragmentos de código en lenguaje natural
- Generan documentación a partir del código
- Sugieren correcciones para errores comunes
- Traducen código entre lenguajes

Ejemplo real: Un estudiante pega un error de Python y el asistente explica qué significa, dónde está el problema y ofrece una solución con ejemplo.

■ 2,2— Casos de uso principales

6. TRADUCCIÓN TÉCNICA

Sistemas que traducen documentación manteniendo:

- Terminología técnica precisa
- Estilo y tono del original
- Consistencia en términos recurrentes
- Formatos de código y marcado intactos

Ejemplo real: Una empresa traduce su documentación de API de inglés a español. La traducción mantiene los nombres de endpoints y parámetros sin traducir, pero explica su funcionamiento en el idioma local.

■ 2,2— Casos de uso principales

7. GENERACIÓN DE CONTENIDO

Aplicaciones que crean contenido automáticamente:

- Descripciones de productos a partir de datos estructurados
- Titulares para artículos de noticias
- Emails personalizados para campañas de marketing
- Informes ejecutivos a partir de datos de dashboards

Ejemplo real: Una tienda online con 10.000 productos genera descripciones únicas para cada uno usando datos de características técnicas, mejorando el SEO sin trabajo manual.

■ 2,3— Flujo típico de NLP en una app

- El usuario envía texto
 - La app lo envía a la API
 - La API procesa el texto
 - Devuelve una respuesta estructurada
 - La app actúa según el resultado
-
- Ejemplo: Texto → API → sentimiento: "positivo" → acción: mostrar recomendación

■ 2,3— Flujo típico de NLP en una app

- El usuario envía texto
 - → Puede ser una opinión, una reseña, un mensaje o una pregunta.
- La aplicación envía ese texto a la API
 - → Se hace una llamada HTTP con el contenido como input.
- La API procesa el texto
 - → Aplica modelos de NLP para analizar tono, intención y emociones.
- Devuelve una respuesta estructurada
 - → Por ejemplo: { "sentimiento": "positivo", "confianza": 0.92 }
- La aplicación actúa según el resultado
 - → Decide qué mostrar, recomendar o activar en función del análisis.

Texto del usuario: "Me encantó la atención, volveré seguro."

→ API devuelve: sentimiento = "positivo"

→ Acción: mostrar recomendación personalizada o pedir reseña pública

■ 2,3— Flujo típico de NLP en una app

EL FLUJO COMPLETO: DE TEXTO A ACCIÓN

El flujo de procesamiento NLP en una aplicación no es simplemente "texto → API → respuesta". Hay capas intermedias, validaciones, transformaciones y acciones que determinan el éxito de la integración.

FASE 1: RECEPCIÓN DEL TEXTO

El usuario interactúa con la aplicación mediante:

- Un campo de texto en una interfaz web/móvil
- Un mensaje de voz transcrito previamente
- Un correo electrónico recibido
- Un documento subido (PDF, Word)
- Un comentario o reseña en la plataforma

■ 2,3— Flujo típico de NLP en una app

FASE 2: PREPROCESAMIENTO

Antes de enviar a la API, puedes aplicar transformaciones:

- Limpiar texto: eliminar HTML, caracteres especiales, espacios redundantes
- Truncar si excede límites de tokens (modelos tienen context window limitado)
- Normalizar: convertir a minúsculas si no afecta al significado
- Extraer contexto adicional: si es una conversación, incluir mensajes previos

```
def preprocesar_texto(texto, max_tokens=4000):  
    # Eliminar HTML  
    import re  
    texto_limpio = re.sub(r'<[^>]+>', '', texto)  
  
    # Eliminar espacios múltiples  
    texto_limpio = re.sub(r'\s+', ' ', texto_limpio).strip()  
  
    # Truncar a max_tokens aproximado (4 caracteres ≈ 1 token)  
    if len(texto_limpio) > max_tokens * 4:  
        texto_limpio = texto_limpio[:max_tokens * 4] + "..."  
  
    return texto_limpio
```

■ 2,3— Flujo típico de NLP en una app

FASE 3: CONSTRUCCIÓN DEL PROMPT

El prompt debe estructurarse para obtener resultados predecibles:

System prompt: define rol, tono, reglas y formato de salida

Contexto: información adicional que el modelo debe considerar

Instrucciones específicas: qué debe hacer exactamente

Ejemplos (few-shot): muestras del formato esperado

Input del usuario: el texto a procesar

■ 2,3— Flujo típico de NLP en una app

FASE 4: LLAMADA A LA API

Se envía la petición con parámetros optimizados:

model: elegir el adecuado (gpt-4o-mini para tareas simples, gpt-4o para complejas)

temperature: baja (0-0.3) para tareas analíticas, más alta para creativas

max_tokens: limitar respuesta para controlar costes

response_format: solicitar JSON cuando sea posible

■ 2,3— Flujo típico de NLP en una app

FASE 5: VALIDACIÓN DE LA RESPUESTA

Antes de usar la respuesta, validar:

- ¿La API respondió con éxito? (código 200)
- ¿El formato es el esperado? (JSON válido si se solicitó)
- ¿Los campos existen y tienen valores coherentes?
- ¿La respuesta está vacía o es demasiado corta?
- ¿Hay indicios de alucinación (información inventada)?

Procesamiento de lenguaje natural aplicado a software.



■ 2,3— Flujo típico de NLP en una app

FASE 6: TRANSFORMACIÓN DE LA RESPUESTA

Adaptar la respuesta al formato que necesita la aplicación:

Convertir JSON a objetos de Python

Mapear categorías a enums internos

Formatear fechas al estándar de la aplicación

Enriquecer con datos adicionales de la base de datos



Procesamiento de lenguaje natural aplicado a software.



■ 2,3— Flujo típico de NLP en una app

FASE 7: ACCIÓN BASADA EN EL RESULTADO

Según el análisis NLP, la aplicación ejecuta acciones:

- Mostrar resultado al usuario
- Redirigir a un flujo específico
- Almacenar en base de datos
- Disparar notificaciones
- Escalar a un agente humano
- Modificar estado interno de la aplicación



2,3— Flujo típico de NLP en una app

EJEMPLO COMPLETO: CLASIFICACIÓN DE TICKET DE SOPORTE

Texto usuario: "Llevo 3 días sin poder acceder a mi cuenta, ya restablecí contraseña 2 veces"

Fase 1: Recibir texto

Fase 2: Preprocesar (limpiar, normalizar)

Fase 3: Construir prompt con system prompt para clasificación JSON

Fase 4: Llamada API → recibe {"categoria": "cuenta", "intencion": "acceso", "urgencia": "alta", "accion": "reset_2fa"}

Fase 5: Validar que todos los campos existen y tienen valores permitidos

Fase 6: Transformar a objeto Ticket con enum Categoria.CUENTA

Fase 7: Acción: escalar a soporte técnico con prioridad alta, enviar alerta, mostrar mensaje "Un agente te contactará en breve"

Procesamiento de lenguaje natural aplicado a software.



Ejemplo en Python: flujo completo

```
```python
=====
FLUJO COMPLETO DE NLP EN UNA APLICACIÓN
=====
import os
import json
import re
from openai import OpenAI
from dotenv import load_dotenv
from datetime import datetime
from enum import Enum

load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

===== FASE 1: MODELOS DE DATOS =====
class CategoriaTicket(Enum):
 CUENTA = "cuenta"
 TECNICO = "tecnico"
 FACTURACION = "facturacion"
 PRODUCTO = "producto"
 OTRO = "otro"

class UrgenciaTicket(Enum):
 ALTA = "alta"
 MEDIA = "media"
 BAJA = "baja"
```

```
class TicketAnalizado:
 """Representa un ticket después del análisis NLP"""
 def __init__(self, texto_original, categoria, urgencia, intencion,
accion_sugerida, entidades=None):
 self.texto_original = texto_original
 self.categoria = categoria
 self.urgencia = urgencia
 self.intencion = intencion
 self.accion_sugerida = accion_sugerida
 self.entidades = entidades or {}
 self.fecha_analisis = datetime.now()

 def to_dict(self):
 return {
 "texto_original": self.texto_original[:100] + "..." if
len(self.texto_original) > 100 else self.texto_original,
 "categoria": self.categoria.value if isinstance(self.categoria,
Enum) else self.categoria,
 "urgencia": self.urgencia.value if isinstance(self.urgencia,
Enum) else self.urgencia,
 "intencion": self.intencion,
 "accion_sugerida": self.accion_sugerida,
 "entidades": self.entidades,
 "fecha_analisis": self.fecha_analisis.isoformat()
 }
```

```
===== FASE 2: PREPROCESAMIENTO
```

```
=====
```

```
def preprocesar_texto(texto: str, max_caracteres: int
= 10000) -> str:
```

```
 """Limpia y normaliza el texto antes del análisis"""
```

```
 # Eliminar HTML
```

```
 texto = re.sub(r'<[^>]+>', '', texto)
```

```
 # Eliminar espacios múltiples
```

```
 texto = re.sub(r'\s+', ' ', texto).strip()
```

```
 # Eliminar caracteres de control
```

```
 texto = re.sub(r'[\x00-\x1f\x7f]', '', texto)
```

```
 # Truncar si es demasiado largo
```

```
 if len(texto) > max_caracteres:
```

```
 texto = texto[:max_caracteres] + "..."
```

```
 return texto
```

# ===== FASE 3: CONSTRUCCIÓN DEL PROMPT =====

SYSTEM\_PROMPT\_CLASIFICACION = """

Eres un sistema de clasificación de tickets de soporte técnico.

Analiza el texto del usuario y devuelve ÚNICAMENTE un JSON con estos campos:

- categoria: una de ["cuenta", "tecnico", "facturacion", "producto", "otro"]
- urgencia: una de ["alta", "media", "baja"]
- intencion: descripción breve de qué quiere hacer el usuario
- accion\_sugerida: qué debería hacer la aplicación (ej: "resetear contraseña", "escalar a tecnico", "reembolsar")
- entidades: objeto con información extraída como { "email": "...", "pedido\_id": "...", "producto": "..." }

Reglas:

- Si el usuario menciona problemas de acceso/login/contraseña → categoria: "cuenta"
- Si menciona errores técnicos, bugs, la app no funciona → categoria: "tecnico"
- Si menciona pagos, facturas, reembolsos → categoria: "facturacion"
- Si menciona productos específicos, características → categoria: "producto"
- Urgencia alta si hay palabras como "urgente", "ya", "inmediatamente", "no funciona"
- No añadas texto fuera del JSON.

"""

```
===== FASE 4: LLAMADA A LA API =====
def analizar_ticket_con_nlp(texto_usuario: str) -> dict:
 """Fase 4: Llama a la API para analizar el ticket"""

 # Fase 2: Preprocesar
 texto_limpio = preprocesar_texto(texto_usuario)

 # Fase 3: Construir prompt
 mensajes = [
 {"role": "system", "content":
SYSTEM_PROMPT_CLASIFICACION},
 {"role": "user", "content": texto_limpio}
]

 try:
 # Fase 4: Llamada a la API
 response = client.chat.completions.create(
 model="gpt-4o-mini",
 messages=mensajes,
 temperature=0.0, # Baja para respuestas
consistentes
 max_tokens=300
)
```

```
Fase 5: Validar respuesta
respuesta_texto = response.choices[0].message.content
```

```
Intentar parsear JSON
try:
 resultado = json.loads(respuesta_texto)
```

```
Validar campos requeridos
campos_requeridos = ["categoria", "urgencia", "intencion", "accion_sugerida"]
for campo in campos_requeridos:
 if campo not in resultado:
 resultado[campo] = "desconocido"
```

```
Validar que categoria es válida
categorias_validas = ["cuenta", "tecnico", "facturacion", "producto", "otro"]
if resultado["categoria"] not in categorias_validas:
 resultado["categoria"] = "otro"
```

```
Validar urgencia
urgencias_validas = ["alta", "media", "baja"]
if resultado["urgencia"] not in urgencias_validas:
 resultado["urgencia"] = "media"
return {
 "exito": True,
 "analisis": resultado,
 "tokens_usados": response.usage.total_tokens
}
```

```
except json.JSONDecodeError:
 # Si no es JSON válido, intentar extraer información de texto libre
 return {
 "exito": False,
 "error": "La API no devolvió JSON válido",
 "respuesta_raw": respuesta_texto
 }
```

```
except Exception as e:
 return {
 "exito": False,
 "error": str(e)
 }
```



```
===== FASE 6: VALIDACIÓN Y TRANSFORMACIÓN =====
def transformar_a_ticket(texto_usuario: str, resultado_api: dict) -> TicketAnalizado:
 """Fase 5-6: Valida y transforma la respuesta en un objeto Ticket"""

 analisis = resultado_api.get("analisis", {})

 # Mapear strings a Enums
 categoria_map = {
 "cuenta": CategoriaTicket.CUENTA,
 "tecnico": CategoriaTicket.TECNICO,
 "facturacion": CategoriaTicket.FACTURACION,
 "producto": CategoriaTicket.PRODUCTO,
 "otro": CategoriaTicket.OTRO
 }

 urgencia_map = {
 "alta": UrgenciaTicket.ALTA,
 "media": UrgenciaTicket.MEDIA,
 "baja": UrgenciaTicket.BAJA
 }

 categoria = categoria_map.get(analisis.get("categoria", "otro"),
 CategoriaTicket.OTRO)
 urgencia = urgencia_map.get(analisis.get("urgencia", "media"), UrgenciaTicket.MEDIA)

 return TicketAnalizado(
 texto_original=texto_usuario,
 categoria=categoria,
 urgencia=urgencia,
 intencion=analisis.get("intencion", "No especificada"),
 accion_sugerida=analisis.get("accion_sugerida", "Revisar manualmente"),
 entidades=analisis.get("entidades", {})
)
```

## # ===== FASE 7: ACCIÓN BASADA EN RESULTADO

=====

```
def ejecutar_accion_segun_ticket(ticket:
TicketAnalizado):
```

```
 """Fase 7: Decide qué hacer según el análisis del
ticket"""
```

```
 print("\n" + "=" * 60)
 print(" ⚡ ACCIONES A EJECUTAR")
 print("=" * 60)
```

```
 # Acciones según categoría y urgencia
```

```
 if ticket.categoria == CategoriaTicket.CUENTA:
```

```
 print(" 🚨 CATEGORÍA: CUENTA")
```

```
 if ticket.urgencia == UrgenciaTicket.ALTA:
```

```
 print(" 🚨 URGENCIA ALTA → Escalar a soporte
inmediatamente")
```

```
 print(" 📧 Enviar alerta al equipo de soporte")
```

```
 print(" 💬 Mostrar al usuario: 'Un agente te
contactará en menos de 5 minutos'")
```

```
 else:
```

```
 print(" 🟡 URGENCIA MEDIA/BAJA → Enviar
email con instrucciones automáticas")
```

```
 print(" 📧 Enviar guía de recuperación de cuenta")
```

```
 elif ticket.categoria == CategoriaTicket.TECNICO:
```

```
 print(" 🚨 CATEGORÍA: TÉCNICO")
```

```
 if ticket.urgencia == UrgenciaTicket.ALTA:
```

```
 print(" 🚨 URGENCIA ALTA → Escalar a equipo técnico con prioridad
máxima")
```

```
 print(" 🛠️ Registrar incidencia en sistema de bugs")
```

```
 else:
```

```
 print(" 🟡 URGENCIA MEDIA/BAJA → Sugerir artículos de la base de
conocimiento")
```

```
 elif ticket.categoria == CategoriaTicket.FACTURACION:
```

```
 print(" 🚨 CATEGORÍA: FACTURACIÓN")
```

```
 print(" 💰 Derivar a departamento de facturación")
```

```
 print(" 📧 Enviar email con instrucciones de pago/reembolso")
```

```
 elif ticket.categoria == CategoriaTicket.PRODUCTO:
```

```
 print(" 🚨 CATEGORÍA: PRODUCTO")
```

```
 print(" 📦 Registrar feedback de producto")
```

```
 print(" 💬 Agradecer al usuario por su comentario")
```

```
 else:
```

```
 print(" 🚨 CATEGORÍA: OTRO")
```

```
 print(" 👤 Escalar a equipo general para revisión manual")
```

```
 # Acción según intención
```

```
 print(f"\n🎯 INTENCIÓN DETECTADA: {ticket.intencion}")
```

```
 print(f"🔧 ACCIÓN SUGERIDA POR IA: {ticket.accion_sugerida}")
```

```
 # Si hay entidades extraídas, mostrarlas
```

```
 if ticket.entidades:
```

```
 print(f"\n📋 ENTIDADES EXTRAÍDAS:")
```

```
 for key, value in ticket.entidades.items():
```

```
 print(f" {key}: {value}")
```

## ■ 2,4— Ejemplo: análisis de sentimiento (Node.js)

```
const response = await client.chat.completions.create({
 model: "gpt-4o-mini",
 messages: [
 {
 role: "user",
 content: "Analiza el sentimiento del texto: 'Estoy muy contento con el servici"
 }
],
 response_format: { type: "json_object" }
});

console.log(response.choices[0].message.content);
```

### ■ ¿Qué hace este código?

- Envía una petición al modelo gpt-4o-mini
- El mensaje del usuario pide analizar el sentimiento
- Se solicita la respuesta en formato JSON

### ■ Ejemplo de flujo completo

Texto del usuario: "Estoy muy contento con el servicio."

→ API devuelve: { "sentimiento": "positivo", "confianza": 0.95 }

→ Acción: mostrar agradecimiento + sugerir dejar reseña pública

```
{ "sentimiento": "positivo", "confianza": 0.95 }
```

## ■ 2,5— Ejemplo: extracción de entidades (Python)

```
1 response = client.chat.completions.create(
2 model="gpt-4o-mini",
3 messages=[
4 {"role": "user",
5 "content": "Extrae nombres, fechas y lugares del texto: 'Juan viajó a Madrid el 3 de marzo.'"}
6],
7 response_format={"type": "json_object"}
8)
9
10 print(response.choices[0].message.content)
11 |
```

### ■ ¿Qué hace este código?

- Envía una petición al modelo gpt-4o-mini
- El mensaje del usuario pide extraer entidades
- Se solicita la respuesta en formato JSON

### ■ Ejemplo de flujo completo

Texto del usuario: "Juan viajó a Madrid el 3 de marzo."

→ API devuelve: { "nombre": "Juan", "lugar": "Madrid", "fecha": "3 de marzo" }

→ Acción: rellenar automáticamente un formulario de viaje o generar una entrada en la agenda.

```
{
 "nombre": "Juan",
 "lugar": "Madrid",
 "fecha": "3 de marzo"
}
```

# Ejemplo en Python: análisis de sentimiento avanzado

```
=====
ANÁLISIS DE SENTIMIENTO AVANZADO
=====

import os
from openai import OpenAI
from dotenv import load_dotenv
import json

load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def analizar_sentimiento_basico(texto: str) -> dict:
 """Nivel básico: solo categoría"""

 response = client.chat.completions.create(
 model="gpt-4o-mini",
 messages=[
 {"role": "system", "content": ""
 Analiza el sentimiento del texto. Responde SOLO con una palabra: positivo, negativo o ne
 utral.
 """},
 {"role": "user", "content": texto}
],
 temperature=0.0
)
```

```
 return {
 "nivel": "básico",
 "sentimiento": response.choices[0].message.content.strip(),
 "texto_original": texto[:100] + "..."
 }

def analizar_sentimiento_intermedio(texto: str) -> dict:
 """Nivel intermedio: polaridad, puntuación, emociones"""

 response = client.chat.completions.create(
 model="gpt-4o-mini",
 messages=[
 {"role": "system", "content": ""
 Analiza el sentimiento del texto. Responde ÚNICAMENTE en formato JSON con:
 - sentimiento: positivo, negativo o neutral
 - polaridad: número entre -1 (muy negativo) y +1 (muy positivo)
 - emociones: objeto con puntuaciones para alegría, tristeza, enojo, sorpresa, miedo
 - intensidad: baja, media, alta
 """},
 {"role": "user", "content": texto}
],
 temperature=0.0
)
```

# Ejemplo en Python: análisis de sentimiento avanzado



```
try:
 resultado = json.loads(response.choices[0].message.content)
 resultado["nivel"] = "intermedio"
 resultado["texto_original"] = texto[:100] + "..."
 return resultado
except:
 return {
 "nivel": "intermedio",
 "error": "No se pudo parsear respuesta",
 "respuesta_raw": response.choices[0].message.content
 }

def analizar_sentimiento_avanzado(texto: str) -> dict:
 """Nivel avanzado: con justificación y fragmentos relevantes"""

 response = client.chat.completions.create(
 model="gpt-4o-mini",
 messages=[
 {"role": "system", "content": ""},
 {"role": "user", "content": f"Analiza el sentimiento del texto en profundidad. Responde ÚNICAMENTE en formato JSON con:
 - sentimiento_global: positivo, negativo o neutral
 - polaridad: número entre -1 y +1
 - fragmentos: lista de objetos con 'texto' y 'sentimiento_individual'
 - justificacion: explicación del análisis
 - tonalidad: formal, coloquial, agresivo, entusiasta, etc.
 - recomendacion: qué acción tomar según el sentimiento
 """}
],
 temperature=0.0
)
```

```
try:
 resultado = json.loads(response.choices[0].message.content)
 resultado["nivel"] = "avanzado"
 resultado["texto_original"] = texto[:100] + "..."
 return resultado
except:
 return {
 "nivel": "avanzado",
 "error": "No se pudo parsear respuesta",
 "respuesta_raw": response.choices[0].message.content
 }

def analizar_sentimiento_multitexto(textos: list) -> list:
 """Analiza sentimiento de múltiples textos y calcula estadísticas"""

 resultados = []
 for texto in textos:
 resultado = analizar_sentimiento_intermedio(texto)
 resultados.append(resultado)

 # Calcular estadísticas agregadas
 polaridades = [r.get("polaridad", 0) for r in resultados if isinstance(r.get("polaridad"), (int, float))]

 estadisticas = {
 "total": len(resultados),
 "positivos": sum(1 for r in resultados if r.get("sentimiento") == "positivo"),
 "negativos": sum(1 for r in resultados if r.get("sentimiento") == "negativo"),
 "neutras": sum(1 for r in resultados if r.get("sentimiento") == "neutral"),
 "polaridad_promedio": sum(polaridades) / len(polaridades) if polaridades else 0
 }
```

# Ejemplo en Python: análisis de sentimiento avanzado

```
===== DEMOSTRACIÓN =====

print("=" * 70)
print("📊 ANÁLISIS DE SENTIMIENTO - COMPARATIVA DE NIVELES")
print("=" * 70)

texto_prueba = "El producto llegó rápido, pero la calidad no es lo que esperaba. La verdad, estoy un poco decepcionado."

print(f"\n📄 Texto a analizar: {texto_prueba}")
print("-" * 70)

print("\n🟢 NIVEL BÁSICO:")
resultado_basico = analizar_sentimiento_basico(texto_prueba)
print(json.dumps(resultado_basico, indent=2, ensure_ascii=False))

print("\n🟡 NIVEL INTERMEDIO:")
resultado_intermedio = analizar_sentimiento_intermedio(texto_prueba)
print(json.dumps(resultado_intermedio, indent=2, ensure_ascii=False))

print("\n🔴 NIVEL AVANZADO:")
resultado_avanzado = analizar_sentimiento_avanzado(texto_prueba)
print(json.dumps(resultado_avanzado, indent=2, ensure_ascii=False))

Análisis de múltiples textos (ejemplo de reseñas)
print("\n" + "=" * 70)
print("📊 ANÁLISIS DE MÚLTIPLES RESEÑAS")
print("=" * 70)
```

```
Análisis de múltiples textos (ejemplo de reseñas)
print("\n" + "=" * 70)
print("📊 ANÁLISIS DE MÚLTIPLES RESEÑAS")
print("=" * 70)

reseñas = [
 "Me encantó este producto, súper recomendado",
 "Regular, cumple pero no es nada del otro mundo",
 "Horrible, no compren esto, es una estafa",
 "Buen producto, buen precio, envío rápido",
 "No me gustó, la calidad es mala"
]

resultado_multiple = analizar_sentimiento_multitexto(reseñas)

print("\n📊 ESTADÍSTICAS AGREGADAS:")
print(f" Total de reseñas: {resultado_multiple['estadisticas']['total']}")
print(f" Positivas: {resultado_multiple['estadisticas']['positivos']}")
print(f" Negativas: {resultado_multiple['estadisticas']['negativos']}")
print(f" Neutrales: {resultado_multiple['estadisticas']['neutrales']}")
print(f" Polaridad promedio: {resultado_multiple['estadisticas']['polaridad promedio']:.2f}")

print("\n📄 RESEÑAS INDIVIDUALES:")
for i, res in enumerate(resultado_multiple['resultados_individuales']):
 print(f"\n Reseña {i+1}: {reseñas[i]}")
 print(f" → Sentimiento: {res.get('sentimiento')} (polaridad: {res.get('polaridad', 'N/A')})")
 if res.get('emociones'):
 emocion_principal = max(res['emociones'].items(), key=lambda x: x[1]) if res['emociones'] else ("ninguna", 0)
 print(f" → Emoción principal: {emocion_principal[0]}")
```

## ■ 2,6— Búsqueda semántica con embeddings

Los **embeddings** convierten texto en vectores numéricos que capturan su **significado contextual**, no solo sus palabras. Esto permite que las aplicaciones trabajen con el contenido de forma más inteligente.

### ¿Qué permiten los embeddings?

- **Buscar documentos similares** Encuentra textos que hablan de lo mismo, aunque usen palabras distintas.  
*Ejemplo:* buscar “problemas de acceso” y encontrar “fallos de autenticación”.
- **Detectar duplicados o contenido redundante** Identifica textos que dicen lo mismo con diferente redacción.
- **Clasificar por contexto** Agrupa textos por tema, tono o intención sin reglas manuales.
- **Construir sistemas RAG (Retrieval Augmented Generation)** Combinan búsqueda semántica + generación de texto. *Ejemplo:* buscar información relevante y generar una respuesta con contexto real.

## ■ 2,6— Búsqueda semántica con embeddings

```
const embedding = await client.embeddings.create({
 model: "text-embedding-3-small",
 input: "Este es el texto a vectorizar"
});

console.log(embedding.data[0].embedding);
```

### ■ ¿Qué hace este código?

Usa el modelo **text-embedding-3-small**

Es rápido, económico y suficiente para tareas de búsqueda semántica y clasificación.

Envía un texto como entrada **"Este es el texto a vectorizar"** será convertido en un vector numérico.

Recibe un **embedding como respuesta** Un array de números que representa el significado del texto. *Ejemplo:* **[0.021, -0.034, 0.112, ...]**

Se imprime el vector en consola Este vector puede almacenarse, compararse o usarse para buscar similitudes.

## 2,7— Tabla: ¿Qué técnica usar?



Necesidad	Técnica NLP	Descripción ampliada	API recomendada
Resumir	<i>Summarization</i>	Reduce textos largos a ideas clave. Útil para informes, artículos, emails extensos o documentación técnica.	<b>GPT-4o</b> (resumen estructurado), <b>Claude</b> (contexto largo).
Detectar intención	<i>Intent classification</i>	Identifica qué quiere hacer el usuario: preguntar, quejarse, pedir soporte, solicitar acción.	<b>Hugging Face BERT</b> , <b>OpenAI</b> (clasificación zero-shot).
Buscar por significado	<i>Embeddings</i>	Convierte texto en vectores para comparar significado, no palabras. Ideal para RAG, FAQs y buscadores inteligentes.	<b>OpenAI Embeddings</b> , <b>Cohere</b> .
Extraer datos	<i>NER (Named Entity Recognition)</i>	Detecta nombres, fechas, lugares, organizaciones o cantidades dentro de un texto.	<b>Hugging Face NER</b> , <b>spaCy</b> , <b>OpenAI</b> (extracción flexible).
Conversar	<i>Chat completions</i>	Genera respuestas coherentes, mantiene contexto y permite interacción natural.	<b>GPT-4o</b> , <b>Claude</b> .
Traducir	<i>Translation</i>	Traduce texto manteniendo tono, intención y precisión técnica.	<b>Google AI</b> , <b>GPT-4o</b> .

## ■ 2,7— Tabla: ¿Qué técnica usar?

### ✓ Resumir

"Cuando el texto es demasiado largo y necesitas una versión corta y clara."

Ejemplo: convertir un PDF de 10 páginas en 5 puntos clave.

### ✓ Detectar intención

"Cuando quieres saber qué quiere hacer el usuario sin que lo diga explícitamente."

Ejemplo: "Necesito ayuda urgente" → intención: *soporte crítico*.

### ✓ Buscar por significado

"Cuando las palabras cambian, pero el concepto es el mismo."

Ejemplo: Buscar "fallo al iniciar sesión" y encontrar documentos que hablan de "error de autenticación".

### ✓ Extraer datos (NER)

"Cuando necesitas sacar información concreta de un texto."

Ejemplo: "Juan viajó a Madrid el 3 de marzo" → nombre, lugar, fecha.

### ✓ Conversar

"Cuando la app debe responder como un asistente."

Ejemplo: chatbots, tutores, soporte técnico.

### ✓ Traducir

"Cuando necesitas contenido en varios idiomas sin perder precisión."

Ejemplo: traducir documentación técnica o mensajes de usuarios.

## ■ 2,8—Buenas prácticas y errores mas comunes

**Definir siempre un *system prompt* claro**

El *system prompt* marca las reglas del asistente: tono, rol, límites y estilo. Un buen *system* evita respuestas ambiguas y reduce errores. *Ejemplo*: "Eres un analista técnico. Responde siempre en JSON."

**Pedir respuestas en formato JSON cuando sea posible**

El JSON permite validar, almacenar y procesar la salida fácilmente. Evita respuestas libres que rompen el flujo de la aplicación.

*Ejemplo*:

```
{ "sentimiento": "positivo", "confianza": 0.92 }
```

**Limitar la creatividad (temperature baja) en tareas analíticas**

Para análisis, extracción o clasificación, la creatividad es un problema. *Temperature 0-0.3* = respuestas más deterministas y fiables.

## 2,8—Buenas prácticas y errores mas comunes

### **Validar siempre la salida antes de usarla**

Nunca confíes ciegamente en la IA. Comprueba que el JSON es válido, que los campos existen y que los valores tienen sentido.

*Ejemplo: verificar que "fecha" realmente es una fecha.*

### **Controlar tokens para evitar costes innecesarios**

Prompts largos = más coste. Respuestas largas = más coste. Optimizar texto, eliminar redundancias y usar modelos pequeños cuando sea posible.

### **Usar embeddings para búsquedas complejas**

No uses chat completions para buscar información. Los embeddings son más rápidos, más baratos y más precisos para similitud semántica.

Procesamiento de lenguaje natural aplicado a software.



## ■ 2,8— y errores mas comunes

### **No definir el rol del modelo**

Sin *system prompt*, el modelo improvisa. Resultado: respuestas inconsistentes, tono variable y errores de formato.

### **Enviar prompts demasiado largos**

Mucho texto = más coste + más ruido. Los programadores suelen copiar/pegar sin filtrar. La clave es *prompt engineering*: claro, corto y específico.

### **No controlar el formato de salida**

Si no pides JSON, el modelo puede devolver texto libre, listas o explicaciones. Esto rompe la lógica de la app. Siempre especificar: "Responde SOLO en JSON".

## ■ 2,8— y errores mas comunes

### Usar modelos grandes para tareas simples

Error típico: usar GPT-4o para clasificar “positivo/negativo”. Modelos pequeños → más rápidos, más baratos y suficientes.

### No validar entidades extraídas

La IA puede inventar fechas, nombres o lugares si el texto es ambiguo. Siempre comprobar:

- ¿La entidad existe?
- ¿Tiene formato correcto?
- ¿Coincide con el texto original?

### No entender cómo funcionan los embeddings

Muchos alumnos creen que “buscar por significado” es lo mismo que “buscar por palabras”. Sin embeddings, la búsqueda semántica no funciona. Es clave entender: *texto* → *vector* → *similitud*.

## EXTRA : Actividad práctica

### Actividad:

Crea un script que:

1. Reciba un texto del usuario
2. Detecte sentimiento
3. Extraiga entidades
4. Devuelva un JSON con:
  - ✓ sentimiento
  - ✓ entidades
  - ✓ resumen

**Objetivo:** Crear un script en Python que analice un texto ingresado por el usuario, detecte su sentimiento, extraiga entidades mencionadas y genere un resumen, devolviendo todo en formato JSON.

### Pasos a seguir:

1. Solicitar al usuario un texto (puede ser una oración, párrafo o artículo).
2. Realizar análisis de sentimiento (positivo, negativo, neutral, o con puntuación).
3. Extraer entidades como personas, organizaciones, lugares, fechas, etc.
4. Generar un breve resumen del texto.
5. Estructurar los resultados en un diccionario y convertirlo a JSON.



Gracias.



MINISTERIO  
DE EDUCACIÓN, FORMACIÓN PROFESIONAL  
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN  
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN



